

VibeMeGood — QC Fix Brief

Issues found after full codebase review. Fix in priority order.

Every item below is a confirmed bug or gap found in the current build. Read this entire document before touching any code.

CRITICAL — Fix First (Runtime Failures or Broken Core Flows)

Fix 1: Slate Board table is unusable on mobile

File: artifacts/prizepicks/src/pages/slate-board.tsx

The table renders 14 columns with zero responsive hiding. On any phone screen it is an unreadable horizontal scroll. This is the most-used page in the app.

What to do: Add `hidden md:table-cell` to non-essential columns in both `<TableHead>` and every matching `<TableCell>` in the row body. Keep visible on all screen sizes: Sport, Player, PP Line, Type, P(Over), Action. Hide on mobile (show on md+): Team, Opp, Mkt Avg, True Edge, Our Proj, Streak. Hide on mobile and tablet (show on lg+): book line detail columns.

```
// TableHead — add hidden class to non-essential columns
<TableHead className="hidden md:table-cell w-12 font-mono text-xs">Team</TableHead>
<TableHead className="hidden md:table-cell w-12 font-mono text-xs">Opp</TableHead>
<TableHead className="hidden lg:table-cell w-16 font-mono text-xs text-right">Mkt
<TableHead className="hidden lg:table-cell w-22 font-mono text-xs text-right">Tru
<TableHead className="hidden lg:table-cell w-28 font-mono text-xs text-right">Our
<TableHead className="hidden md:table-cell w-14 font-mono text-xs text-center">St

// Apply the same hidden class to every matching TableCell in the row body
<TableCell className="hidden md:table-cell font-mono text-xs text-muted-foreground">
<TableCell className="hidden md:table-cell font-mono text-xs text-muted-foreground">
// etc.
```

Acceptance: On a 390px wide screen (iPhone), the table shows: player name, stat, PP line, type badge, P(Over), and action button — all readable without horizontal scrolling.

Fix 2: Force Sync does not refresh projections

File: artifacts/api-server/src/routes/sync.ts

The `/api/sync/all` endpoint only queues `pp-lines` and `external-odds`. After clicking Force Sync, projections are never recalculated. This means the Slate Board shows stale P(Over) and Our Proj values even after a fresh line pull.

What to do: Add projections and variance as sequential steps in `/sync/all`:

```
// In the jobs array inside router.post("/sync/all"):
const jobs = [
  { name: "pp-lines",      provider: "prizepicks",    fn: syncPpLines },
  { name: "external-odds", provider: "the-odds-api",  fn: syncExternalOdds },
  { name: "projections",   provider: "nba-stats",     fn: syncProjectionsImpl },
];
// syncProjectionsImpl already exists in this file — just add it to the array
```

Acceptance: After clicking Force Sync, the Slate Board shows updated P(Over) values within 30 seconds. The SSE stream broadcasts `projections: success` after `external-odds: success`.

Fix 3: No database transactions on result resolution

File: artifacts/api-server/src/routes/entries.ts

The PATCH handler for entry picks makes 4+ sequential database writes (update pick result, insert CLV record, update calibration, check entry completion) with no transaction wrapper. If the server crashes or an error occurs mid-way, history corrupts — a pick can be marked HIT with no CLV record, or calibration can increment without the pick result being saved.

What to do: Wrap the entire pick resolution block in a Drizzle transaction:

```
// In the PATCH /entry-picks/:id handler, replace sequential awaits with:
await db.transaction(async (tx) => {
  // 1. Update pick result
  await tx.update(entryPicksTable)
    .set({ result: newResult, updatedAt: new Date() })
    .where(eq(entryPicksTable.id, pickId));

  // 2. Insert CLV record (if closing line exists)
  if (closingLine) {
    await tx.insert(clvRecordsTable).values({ ... });
  }
});
```

```

    }

    // 3. Update probability calibration
    await recalibrateProbability(tx, pickId, newResult);

    // 4. Update player streak
    await updatePlayerStreak(tx, pickId, newResult);

    // 5. Recalculate entry result if all picks resolved
    await maybeResolveEntry(tx, entryId);
  });

```

Extract `recalibrateProbability`, `updatePlayerStreak`, and `maybeResolveEntry` as functions that accept a `tx` parameter (Drizzle transaction instance) so they can be called within or outside the transaction.

Acceptance: If you kill the server mid-result-mark and restart, no partial state exists. Either all fields updated or none did.

Fix 4: No Zod validation on any backend route

Files: `artifacts/api-server/src/routes/entries.ts`, `sync.ts`, `anthropic.ts`, and any route that reads `req.body`

Zero Zod validation exists across all backend routes. Bad payloads return 500 errors instead of 400 and can write garbage to the database.

What to do: Add Zod schemas and `safeParse` to the highest-risk routes at minimum. Install Zod if not present (`pnpm add zod`).

```

import { z } from "zod";

// Entry creation
const CreateEntrySchema = z.object({
  stake: z.number().positive().max(10000),
  entryType: z.enum(["power", "flex"]),
  picks: z.array(z.object({
    ppLineId: z.number().int().positive(),
    direction: z.enum(["more", "less"]),
  })).min(2).max(6),
  notes: z.string().max(500).optional(),
});

// Pick result marking

```

```

const PickResultSchema = z.object({
  result: z.enum(["hit", "miss", "dnp"]),
  closingLine: z.number().positive().optional(),
});

// Settings update
const UserSettingsSchema = z.object({
  bankroll: z.number().positive().optional(),
  unitSize: z.number().positive().optional(),
  kellyFraction: z.number().min(0.05).max(1).optional(),
  dailyLossLimit: z.number().positive().nullable().optional(),
  // ... other fields
});

// Usage pattern in every route:
const parsed = CreateEntrySchema.safeParse(req.body);
if (!parsed.success) {
  return res.status(400).json({ error: "Invalid input", issues: parsed.error.issues });
}
// Use parsed.data instead of req.body from here on

```

Apply Zod validation to: POST /entries, PATCH /entry-picks/:id, PUT /user-settings, POST /sync/* routes, POST /anthropic/conversations/:id/messages.

Acceptance: Sending { stake: -999, picks: "not-an-array" } to POST /entries returns 400 with a readable issues array, not a 500.

Fix 5: Auth uses client-provided userId — not Clerk

Files: artifacts/api-server/src/routes/entries.ts and any route using

```
req.query.userId ?? "local"
```

Every protected route currently reads the user ID from the query string: `req.query.userId ?? "local"`. This means any client can access or modify any user's data by passing a different `userId`. Clerk is installed and configured but its auth middleware is not being applied.

What to do:

Step 1 — Apply Clerk Express middleware in `app.ts`:

```

import { clerkMiddleware } from "@clerk/express";
// Add before the router, after cors():
app.use(clerkMiddleware());

```

Step 2 — Add `requireAuth()` to protected route groups:

```
import { requireAuth } from "@clerk/express";
app.use("/api/entries", requireAuth());
app.use("/api/user-settings", requireAuth());
app.use("/api/clv", requireAuth());
app.use("/api/sync", requireAuth()); // prevent external sync triggers
```

Step 3 — Replace `req.query.userId ?? "local"` with `req.auth?.userId`:

```
// Before (insecure):
const userId = (req.query.userId as string) ?? "local";

// After (Clerk-secured):
const userId = req.auth?.userId;
if (!userId) return res.status(401).json({ error: "Unauthorized" });
```

Apply this replacement everywhere `req.query.userId` or `req.body.userId` appears in protected routes. Remove `userId` from all request payloads — the backend derives it from the authenticated session.

Acceptance: Calling `GET /api/entries` without a valid Clerk session returns 401. A logged-in user cannot access entries belonging to a different `userId` by manipulating the request.

MEDIUM — Fix After Critical Items

Fix 6: AI chat uses hardcoded base URL

File: `artifacts/prizepicks/src/pages/ai-chat.tsx`

Line near the top: `const BASE = "/api";`

Every other page in the app uses:

```
const base = (import.meta.env.BASE_URL as string).replace(/\$/ , "");
```

The hardcoded `/api` will break if the app is ever served from a subpath (which Replit can do).

Fix:

```
// Replace:
const BASE = "/api";

// With:
const BASE = `${(import.meta.env.BASE_URL as string).replace(/\/$/, "")}/api`;
```

Apply to every `fetch` call in `ai-chat.tsx` that uses `BASE`.

Fix 7: Injuries sync is a stub — surface the gap clearly

File: `artifacts/api-server/src/routes/sync.ts`

`syncInjuriesImpl` returns 0 and does nothing. The Injuries & News page will be permanently empty. This is a known data gap (no free injury API is integrated yet), but the user needs to know why, not just see an empty list.

What to do: Update the Injuries & News page empty state to explain the gap:

```
// In injuries.tsx empty state:
<div className="text-center py-16 space-y-3">
  <Activity className="w-10 h-10 text-muted-foreground/40 mx-auto" />
  <div className="text-sm font-medium text-muted-foreground">No injury data yet</div>
  <div className="text-xs text-muted-foreground/70 max-w-sm mx-auto">
    Automated injury sync requires a connected feed (Rotowire or ESPN).
    You can manually add injury notes using the button below,
    or this page will populate once an injury feed is connected.
  </div>
  <Button size="sm" variant="outline">Add Manual Injury Note</Button>
</div>
```

Also add a manual injury entry form so the page is useful now, before the feed is connected.

Fix 8: Mark Results is in Journal — add a pointer in Entry Builder

Files: `artifacts/prizepicks/src/pages/entry-builder.tsx`

The Mark Results workflow (hit/miss/DNP per pick) lives in the Journal page, which is correct but discoverable only if users navigate there. Users who just submitted an entry from Entry Builder won't know to go to Journal to grade their picks.

What to do: Add a "Mark Results" callout at the bottom of Entry Builder when there are pending (unresolved) entries:

```
// Add after the picks list in entry-builder.tsx:
{pendingEntryCount > 0 && (
  <div className="p-3 bg-amber-950/30 border border-amber-700/40 rounded-lg flex
    <div>
      <div className="text-xs font-semibold text-amber-300">
        {pendingEntryCount} pending {pendingEntryCount === 1 ? "entry" : "entries"}
      </div>
      <div className="text-xs text-amber-200/60">
        Grade your picks in Journal to track CLV and calibrate your model
      </div>
    </div>
    <Link href="/journal">
      <Button size="sm" variant="outline" className="border-amber-700/50 text-amber-300">
        Mark Results →
      </Button>
    </Link>
  </div>
)}
```

Query pending entries from `/api/entries?result=pending&limit=1` to get the count.

UX / MOBILE IMPROVEMENTS — After Critical and Medium

Fix 9: Slate Board filter row is unusable on mobile

File: `artifacts/prizepicks/src/pages/slate-board.tsx`

The filter controls (sport filter, line type filter, stat type dropdown, search input, Force Sync button) are in a `flex flex-wrap` row. On mobile this becomes 3–4 stacked rows of controls taking up half the screen before any data appears.

What to do: On mobile, collapse filters to a single “Filters” button that opens a bottom drawer. Keep full filter row visible on `md:` and above.

```
const [filterOpen, setFilterOpen] = useState(false);

// Count active filters for badge
const activeFilterCount = [sport, lineTypeFilter, statTypeFilter, minEdge, search
  .filter(Boolean)].length;

return (
  <>
```

```

    { /* Mobile: single filter button */}
    <div className="md:hidden flex items-center gap-2">
      <Button size="sm" variant="outline" onClick={() => setFilterOpen(true)} cla
      <Filter className="w-3.5 h-3.5" />
      Filters
      {activeFilterCount > 0 && (
        <span className="bg-primary text-primary-foreground rounded-full w-4 h-
          {activeFilterCount}
        </span>
      )}
    </Button>
    <ForceSyncButton />
  </div>

  { /* Desktop: full filter row */}
  <div className="hidden md:flex gap-3 flex-wrap items-center">
    { /* existing filter controls */}
    <ForceSyncButton />
  </div>

  { /* Mobile filter drawer */}
  <Sheet open={filterOpen} onOpenChange={setFilterOpen}>
    <SheetContent side="bottom" className="h-auto pb-8">
      <SheetHeader><SheetTitle className="font-mono text-sm">Filters</SheetTitl
      <div className="space-y-4 mt-4">
        { /* All filter controls, full width, stacked */}
      </div>
    </SheetContent>
  </Sheet>
</>
);

```

Fix 10: Force Sync button gives no intermediate feedback

File: artifacts/prizepicks/src/pages/slate-board.tsx — ForceSyncButton component

The button spins for 15 seconds then refreshes. Users cannot tell if it succeeded, which job is running, or if it hung.

What to do: Subscribe to the SSE `sync_status` events (already broadcast by the server) and show which job is currently running inline under the button:

```

function ForceSyncButton() {
  const [syncing, setSyncing] = useState(false);
  const [syncStep, setSyncStep] = useState<string | null>(null);

```

```

const { toast } = useToast();
const qc = useQueryClient();
const base = (import.meta.env.BASE_URL as string).replace(/\/$/, "");

useEffect(() => {
  if (!syncing) return;
  const es = new EventSource(`${base}/api/events`);
  es.addEventListener("sync_status", (e) => {
    const { job, status } = JSON.parse(e.data) as { job: string; status: string };
    if (status === "running") setSyncStep(`${job}...`);
    if (job === "all" && status === "success") {
      setSyncing(false);
      setSyncStep(null);
      qc.invalidateQueries();
      es.close();
    }
    if (status === "error") setSyncStep(`${job} failed`);
  });
  return () => es.close();
}, [syncing, base, qc]);

async function forceSync() {
  setSyncing(true);
  setSyncStep("starting...");
  try {
    await fetch(`${base}/api/sync/all`, { method: "POST" });
  } catch {
    toast({ title: "Sync failed", variant: "destructive" });
    setSyncing(false);
  }
}

return (
  <div className="flex flex-col items-end gap-0.5">
    <Button size="sm" variant="outline" onClick={forceSync} disabled={syncing}
      className="gap-1.5 font-mono text-xs border-primary/30 text-primary hover
        <RefreshCw className={`w-3 h-3 ${syncing ? "animate-spin" : ""}`} />
      {syncing ? "Syncing..." : "Force Sync"}
    </Button>
    {syncStep && (
      <span className="text-[10px] font-mono text-muted-foreground">{syncStep}<
    )}
  </div>
);
}

```

ACCEPTANCE TEST — Run All 14 After Fixes

1. Force Sync → real player names appear in Slate Board within 30 seconds.
SSE shows: pp-lines → external-odds → projections (sequential, labeled).
2. 2+ props show DK and FD lines when ODDS_API_KEY is set.
3. 5+ NBA props show OUR PROJ with confidence badge.
4. MLB prop shows pitcher name in projection notes.
5. Outdoor MLB game shows weather badge. Domed stadium (TOR, TBR, MIA) shows non
6. Optimizer / Goblin Hunter / 4-pick Power → all 4 are Goblin OVER.
7. Entry Builder shows probability chain and Kelly stake for 4-pick entry.
8. Add 2 NBA teammates → correlation warning appears, names the team.
9. AI Analyst / Contradiction Check → Claude names real players from today's sla
10. Mark a pick HIT with closing line 27.5 → CLV record appears in CLV Tracker.
11. GET /api/debug/calibration?sport=NBA&statType=Points → returns sampleSize, hitRate, and source label ("Default Estimate" or "Your Data").
12. Set \$50 daily loss limit → submit \$50 entry → mark it loss → submit another e → loss limit banner appears.
13. User Guide page loads with TOC and search working.
14. Slate Board on 390px wide phone screen: player name, stat, line, and action are all visible without horizontal scrolling.

SUMMARY — Priority Order

CRITICAL (do these first):

1. Mobile Slate Board — add hidden md:table-cell to non-essential columns
2. Force Sync — add projections to /sync/all job list
3. DB transactions — wrap result resolution in db.transaction()
4. Zod validation — add to entries, result marking, settings, sync routes
5. Clerk auth — apply clerkMiddleware(), replace req.query.userId pattern

MEDIUM (after critical):

6. AI chat hardcoded BASE URL → use `import.meta.env.BASE_URL`
7. Injuries page – add manual entry form and clear empty state explanation
8. Entry Builder – add "Mark Results" pointer for pending entries

UX (after medium):

9. Slate Board mobile filter drawer
10. Force Sync with live step labels via SSE